

Team members -

Aarush Narang

Khushi Chavan

Research Question -

What is the impact of mental health status on employee productivity across different industries?

```
In [1]: import pandas as pd
import numpy as np
import glob
import os
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error, r2_score
```

```
In [4]: directory_hps = 'HPS'
os.chdir(directory_hps)
all_filenames_hps = glob.glob("*.csv")
combined_hps_df = pd.concat([pd.read_csv(f) for f in all_filenames_hps], ignore_index=True)
os.chdir('../')
combined_hps_df.to_csv("combined_hps.csv", index=False)
```

```
In [5]: # Load your full dataset
df = pd.read_csv('combined_hps.csv', low_memory=False)
```

```
In [6]: # List of key factors
columns_needed = [
    'WRKLOSSRV', # Remote work & productivity
    'ANXIOUS', 'WORRY', 'INTEREST', 'DOWN', # Mental health indicators
    'SCRAM' # unique ID
]

# Make a new DataFrame with only selected columns
df_selected = df[columns_needed]

# Save it to a new CSV file
df_selected.to_csv('HPS_selected_columns.csv', index=False)

print("Relevant columns saved")
```

Relevant columns saved

```
In [7]: # Sample data loading – replace with your actual CSV file path
data = pd.read_csv('HPS_selected_columns.csv')

# Replace special missing value codes (-88, -99) with NaN
missing_value_codes = [-88, -99]
data.replace(missing_value_codes, np.nan, inplace=True)

# Remove duplicate rows if any
data.drop_duplicates(inplace=True)

# Fix inconsistencies
# For categorical columns, convert categorical columns to 'category' dtype
categorical_columns = ['WRKLOSSRV',
                       'ANXIOUS', 'WORRY', 'INTEREST', 'DOWN']

for col in categorical_columns:
    if col in data.columns:
        data[col] = data[col].astype('category')

# After cleaning, you can check how many missing values remain per column
missing_summary = data.isnull().sum()

print("Missing values per column:")
print(missing_summary)

# Save cleaned data if needed
data.to_csv('cleaned_dataset.csv', index=False)
```

```
Missing values per column:
WRKLOSSRV    10031
ANXIOUS      38324
WORRY        39270
INTEREST     39300
DOWN         39347
SCRAM         0
dtype: int64
```

```
In [8]: # Load your dataset
data = pd.read_csv('cleaned_dataset.csv')

# Replace empty strings and specific NA placeholders with np.nan, if any
data.replace(['', 'NA', 'NaN'], np.nan, inplace=True)

# Dropping the rows where there are missing values in productivity
data.dropna(subset=['WRKLOSSRV'], inplace=True)

# Dropping the rows where there are missing values in all the mental health t
data.dropna(subset=['ANXIOUS', 'WORRY', 'INTEREST', 'DOWN'], how='all', inpl
```

```
In [9]: data_missing = data[data.isna().any(axis=1)]

data.dropna(subset=['ANXIOUS', 'WORRY', 'INTEREST', 'DOWN'], inplace=True)

print(data.isnull().sum())
```

```
WRKLOSSRV    0
ANXIOUS      0
WORRY        0
INTEREST     0
DOWN         0
SCRAM        0
dtype: int64
```

```
In [10]: cols = ['ANXIOUS', 'WORRY', 'INTEREST', 'DOWN']

row_mean = data_missing[cols].mean(axis=1, skipna=True)

import numpy as np
rounded = np.where(
    row_mean - np.floor(row_mean) >= 0.5,
    np.ceil(row_mean),
    np.floor(row_mean)
)

rounded_s = pd.Series(rounded, index=data_missing.index)

for c in cols:
    data_missing[c] = data_missing[c].fillna(rounded_s)

data.update(data_missing)
print(data.isnull().sum())
```

```
WRKLOSSRV    0
ANXIOUS      0
WORRY        0
INTEREST     0
DOWN         0
SCRAM        0
dtype: int64
```

```
/var/folders/gk/m6_7mj810mq3yvqdk1_1j5qr0000gn/T/ipykernel_24803/1924331079.
py:15: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
    data_missing[c] = data_missing[c].fillna(rounded_s)
```

```
In [11]: data['WRKLOSSRV'] = data['WRKLOSSRV'].replace(2, 0)

cols = ['ANXIOUS', 'WORRY', 'DOWN']
mapping = {1: 0, 2: 1, 3: 1, 4: 1}

for c in cols:
    data[c] = data[c].replace(mapping)

data['INTEREST'] = data['INTEREST'].replace({1: 1, 2: 0, 3: 0, 4: 0})
```

```
In [12]: data.to_csv('imputed_dataset.csv', index=False)
```

```
In [13]: imputed_data = pd.read_csv('imputed_dataset.csv')
print(imputed_data.head())
```

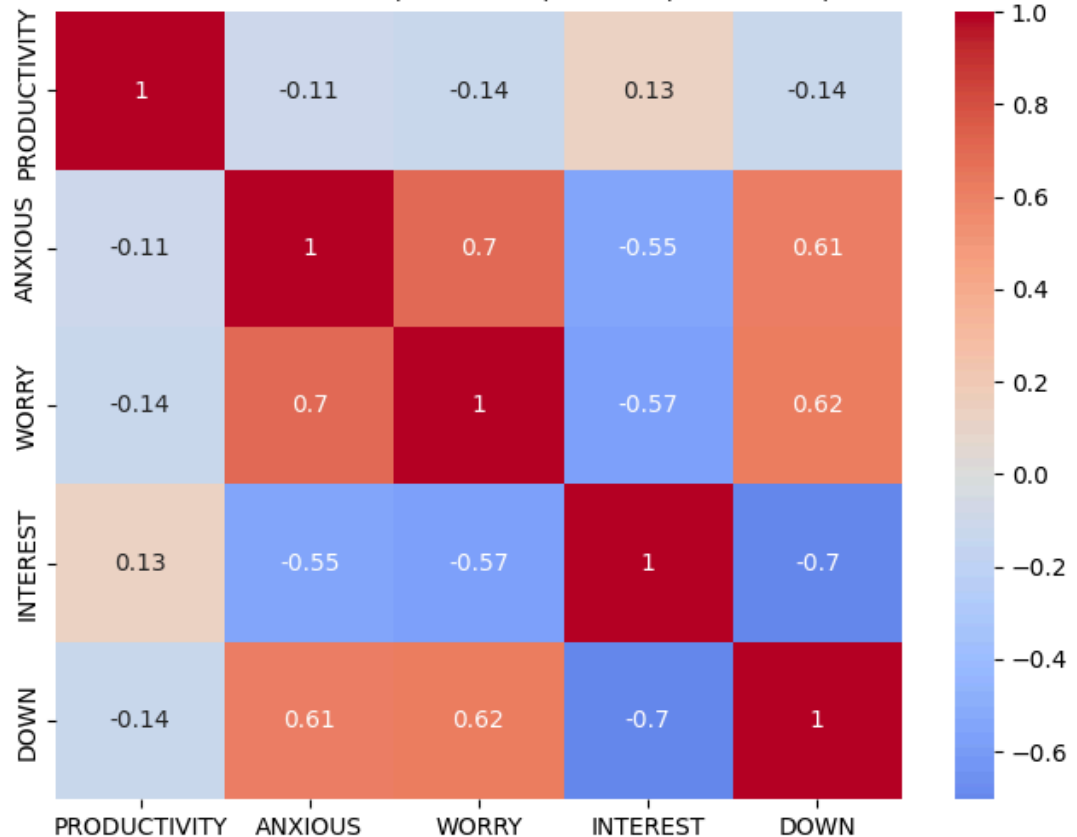
	WRKLOSSRV	ANXIOUS	WORRY	INTEREST	DOWN	SCRAM
0	0.0	0.0	0.0	1.0	0.0	P030000001
1	0.0	0.0	0.0	1.0	0.0	P030000002
2	0.0	0.0	0.0	1.0	0.0	P030000003
3	0.0	0.0	0.0	1.0	0.0	P030000004
4	0.0	0.0	0.0	0.0	0.0	P030000005

```
In [14]: imputed_data['PRODUCTIVITY'] = 1 - imputed_data['WRKLOSSRV']
```

```
In [15]: imputed_data = imputed_data[['PRODUCTIVITY', 'ANXIOUS', 'WORRY', 'INTEREST',
corr_matrix = imputed_data.corr()

plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix of ' + ', '.join(imputed_data.columns) + ' Variables')
plt.show()
```

Correlation Matrix of PRODUCTIVITY, ANXIOUS, WORRY, INTEREST, DOWN Variables



```
In [16]: def data_exploration(data):

    summary_stats = data.describe(include='all')
    print("Statistical Summary:")
    print(summary_stats)

    mental_health_metrics = ['ANXIOUS', 'WORRY', 'DOWN']
    for metric in mental_health_metrics:
```

```
plt.figure(figsize=(8,5))
sns.boxplot(data=data, x='PRODUCTIVITY', y=metric)
plt.title(f'{metric} Distribution by Work Type')
plt.show()

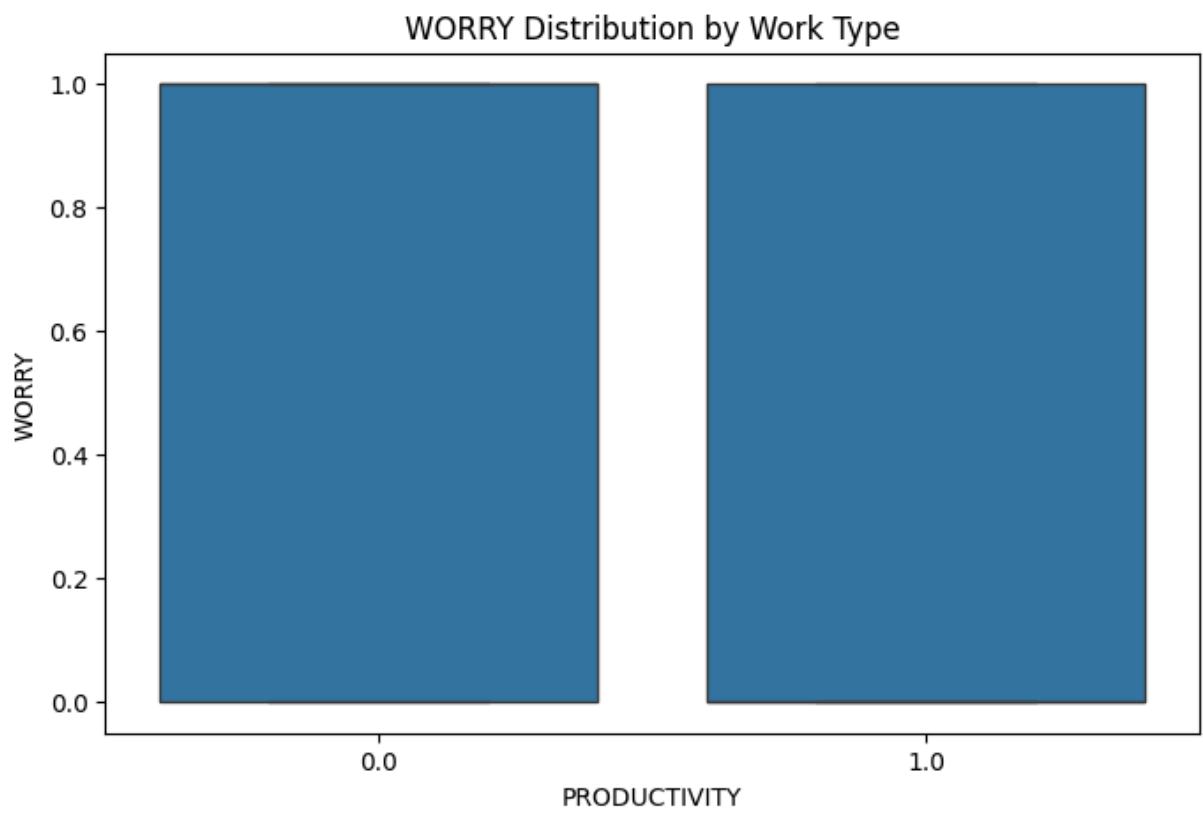
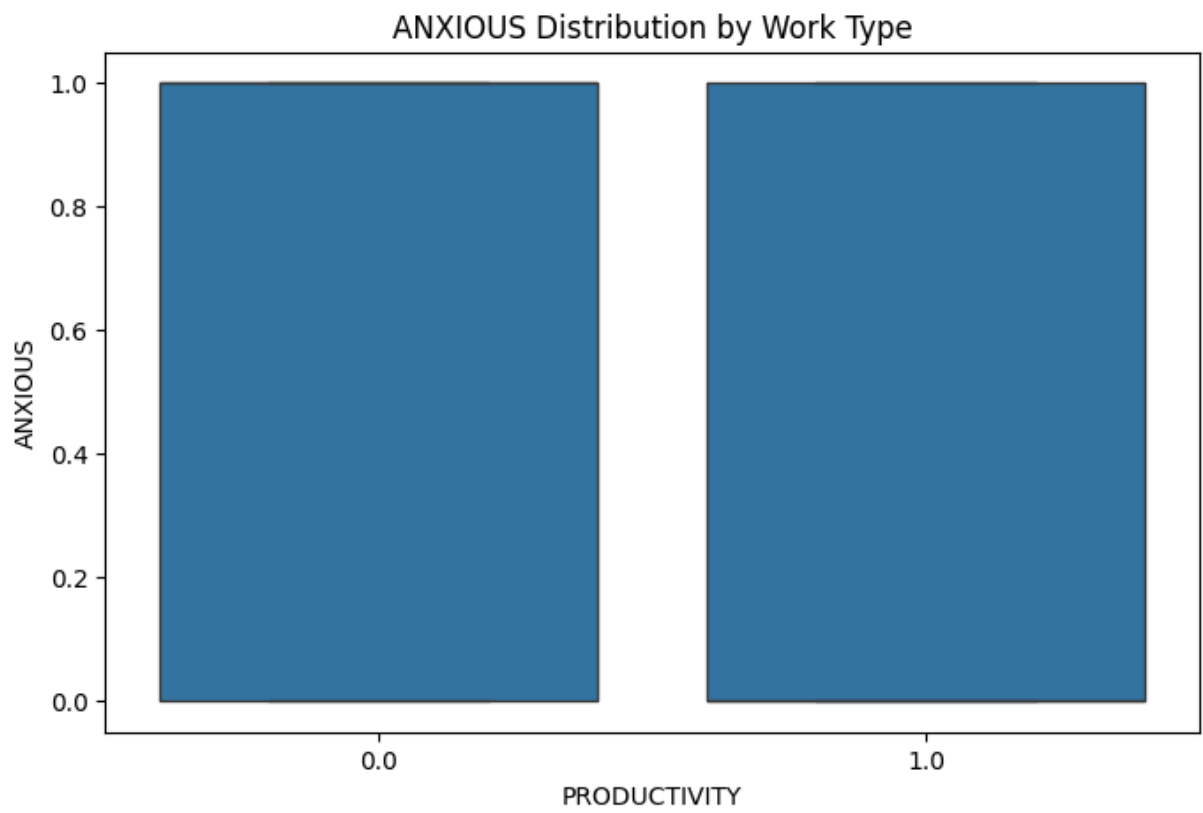
for metric in mental_health_metrics:
    plt.figure(figsize=(8,5))
    sns.boxplot(data=data, x='INTEREST', y=metric)
    plt.title(f'{metric} Distribution by Work Type')
    plt.show()
```

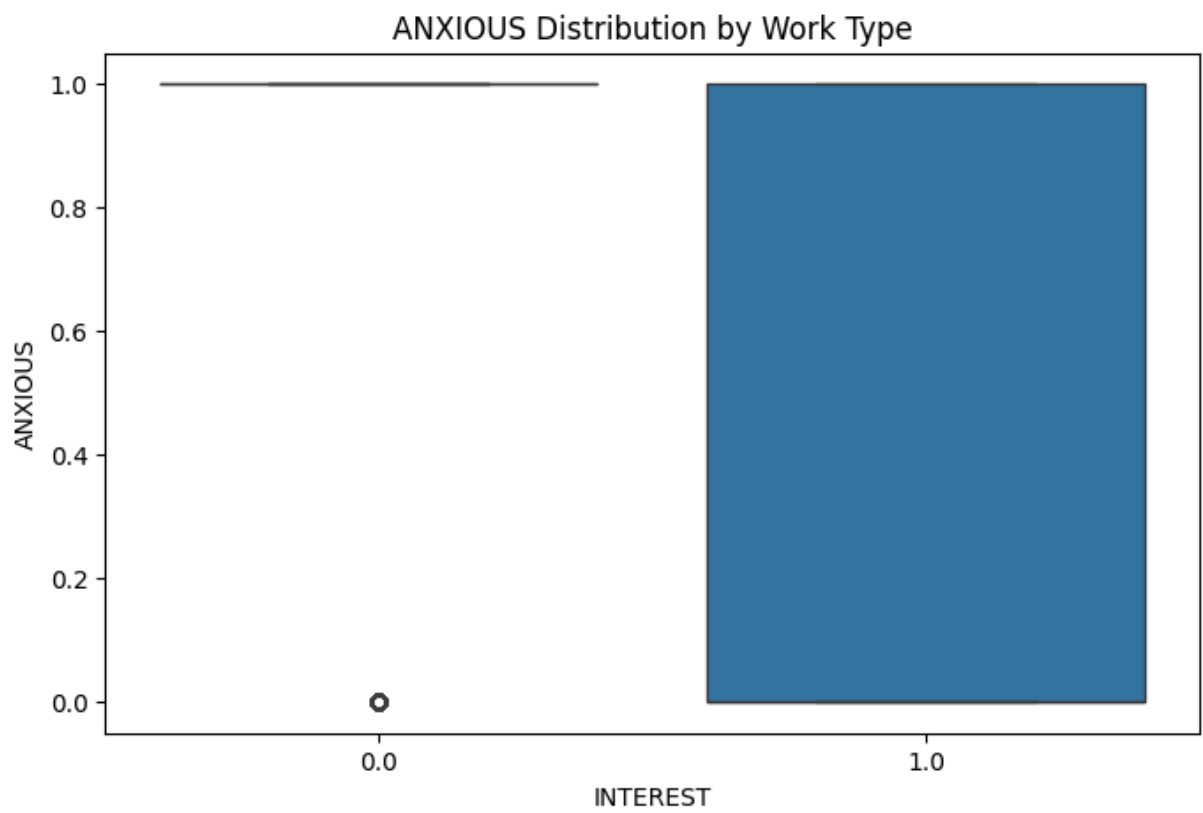
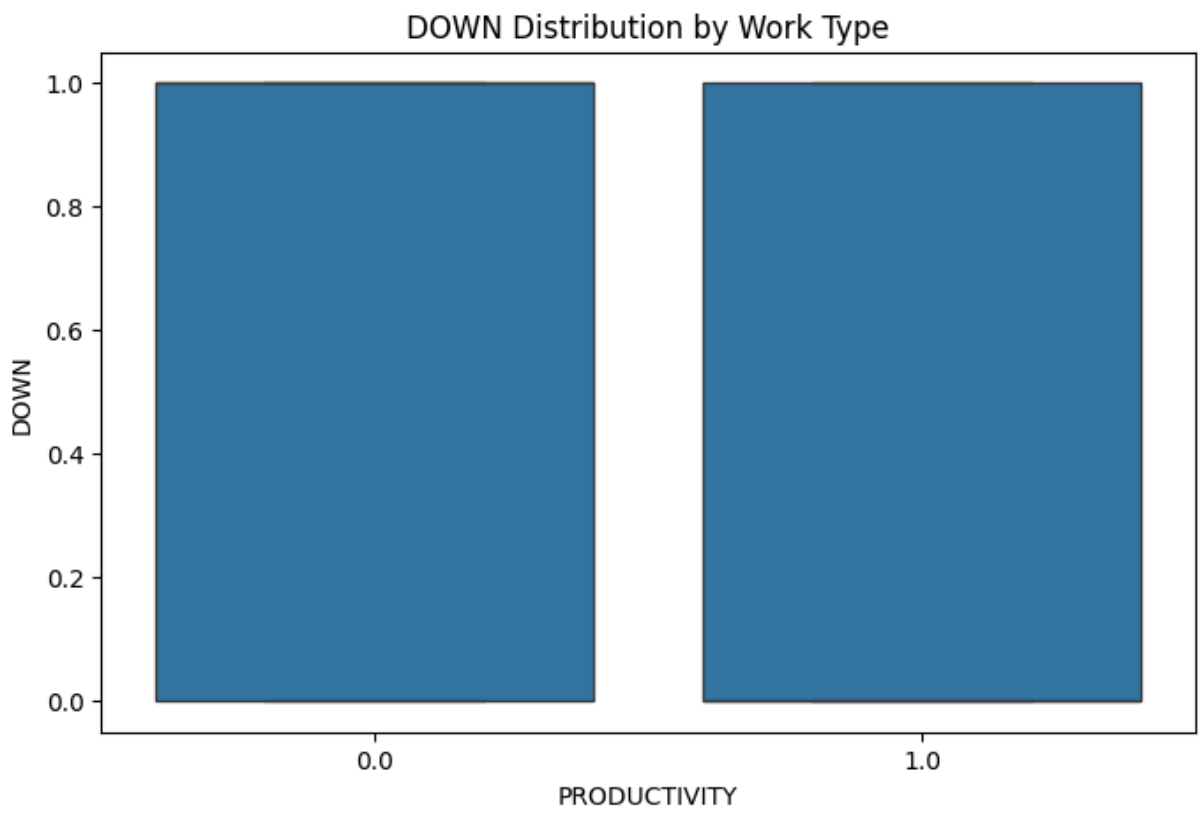
```
data_exploration(imputed_data)
```

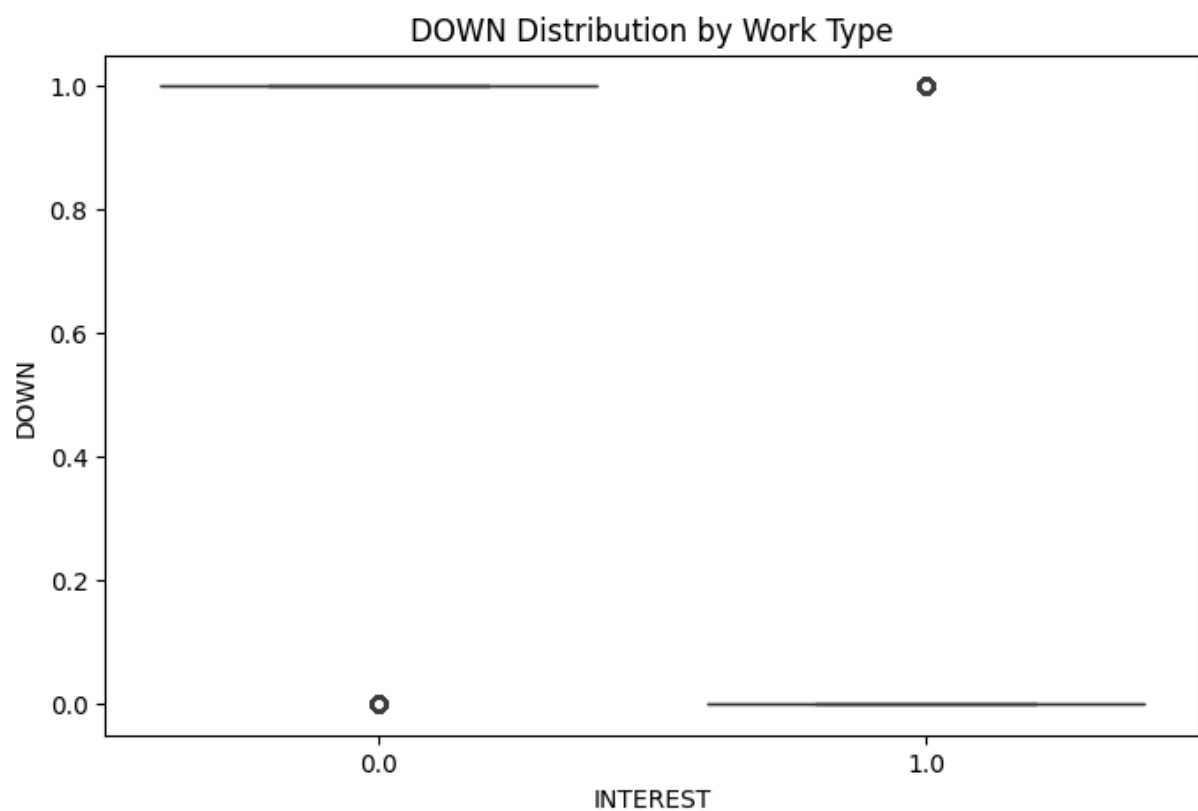
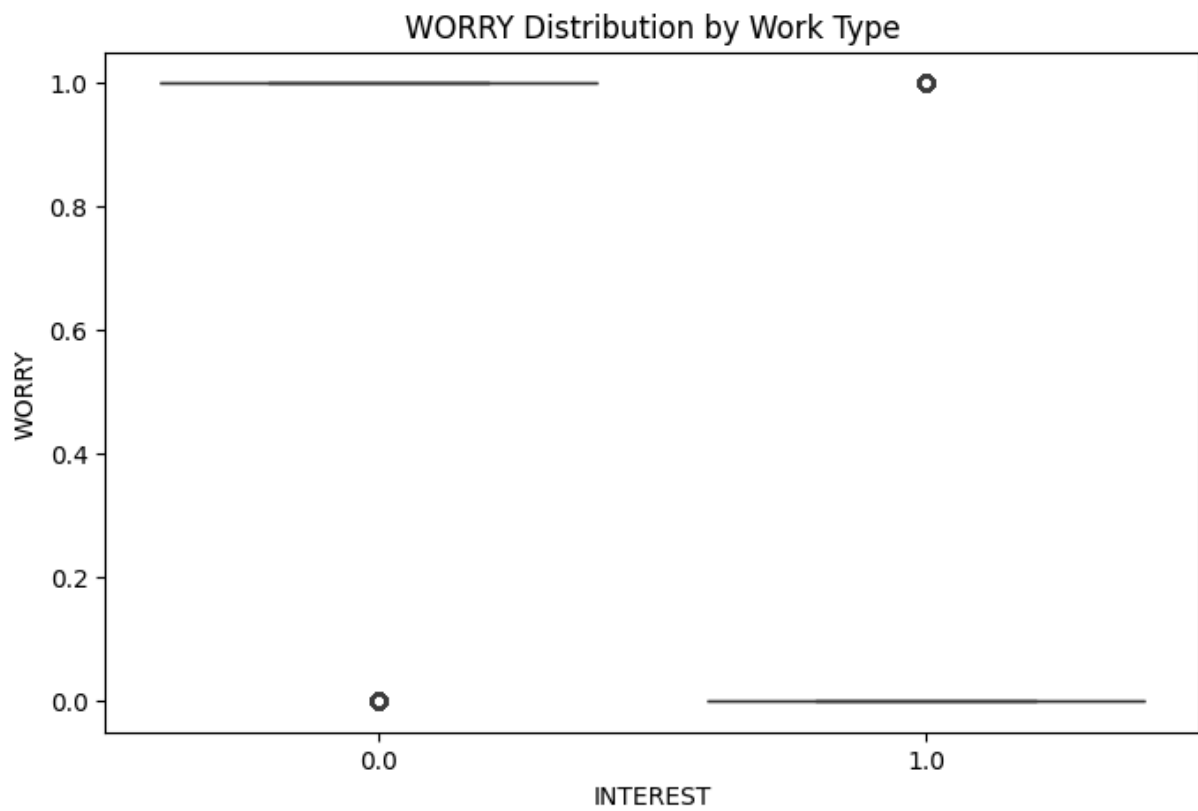
Statistical Summary:

	PRODUCTIVITY	ANXIOUS	WORRY	INTEREST \
count	568282.000000	568282.000000	568282.000000	568282.000000
mean	0.903441	0.434647	0.343368	0.691255
std	0.295357	0.495711	0.474834	0.461976
min	0.000000	0.000000	0.000000	0.000000
25%	1.000000	0.000000	0.000000	0.000000
50%	1.000000	0.000000	0.000000	1.000000
75%	1.000000	1.000000	1.000000	1.000000
max	1.000000	1.000000	1.000000	1.000000

	DOWN
count	568282.000000
mean	0.319607
std	0.466325
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	1.000000







```
In [ ]: # Load dataset and select columns
selected_features = ['ANXIOUS', 'WORRY', 'INTEREST', 'DOWN', 'PRODUCTIVITY']

selected_data = imputed_data[selected_features]

X = selected_data.drop(['PRODUCTIVITY'], axis=1)
```



```

y_prod = selected_data['PRODUCTIVITY']

# Encode categoricals for regression
# X = pd.get_dummies(X, drop_first=True)

# Train/validation/test splits (70/15/15)
X_train, X_temp, y_prod_train, y_prod_temp = train_test_split(
    X, y_prod, test_size=0.3, random_state=42
)
X_val, X_test, y_prod_val, y_prod_test = train_test_split(
    X_temp, y_prod_temp, test_size=0.5, random_state=42
)

lr_prod = LinearRegression()
lr_prod.fit(X_train, y_prod_train)

val_prod_pred = lr_prod.predict(X_val)

print("Validation (INTEREST): R2 =", r2_score(y_prod_val, val_prod_pred),
      "RMSE =", root_mean_squared_error(y_prod_val, val_prod_pred))

test_prod_pred = lr_prod.predict(X_test)

print("Test (INTEREST): R2 =", r2_score(y_prod_test, test_prod_pred),
      "RMSE =", root_mean_squared_error(y_prod_test, test_prod_pred))

```

Validation (INTEREST): R2 = 0.526760809965384 RMSE = 0.3179373859478403
 Test (INTEREST): R2 = 0.5329614677167669 RMSE = 0.31511270833848143